Preference-Aware Deep Reinforcement Learning with GP-Evolved Actions for Dynamic Workflow Scheduling in Cloud-Edge Computing

Zaixing Sun, Kaixuan Ma, Bin Wang, Meng Xu, Chonglin Gu, and Hejiao Huang

***Abstract*—Online workflow scheduling in heterogeneous Cloud–Edge systems must allocate resources before future workflow arrivals are known. Resource choices have state-dependent effects on both workflow tardiness and energy consumption. Edge Nodes avoid Cloud provisioning and reduce wide-area communication delays but have limited capacity, whereas Cloud Servers provide elastic and powerful computing capacity but incur additional provisioning and communication overheads. Their relative benefits further depend on task characteristics, workflow dependencies, and energy efficiency. Scheduling policies must therefore adapt to user-requested objective trade-offs and evolving system states. Genetic programming (GP) hyper-heuristics have demonstrated effectiveness in evolving resource-selection rules for dynamic workflow scheduling, but usually apply them as fixed policies. Deep reinforcement learning enables adaptive control, yet directly treating a variable set of feasible resources as actions while handling multiple preferences complicates policy learning. We propose Preference-aware Risk-Stratified Genetic Programming-assisted Deep Reinforcement Learning (PRS-GPDRL). Cooperative coevolution genetic programming first evolves coupled task- and resource-selection rules. Quality- and behavior-aware filtering then retains a compact resource-selection rule pool from the final CCGP archive. Each retained rule is paired with base and risk modes. A multi-objective double deep Q-network then selects a rule-mode action under the requested preference at each allocation decision. Across 12 scenarios (30 independent runs each), PRS-GPDRL achieves the best average ranks (HV: 1.406; IGD: 1.478) and statistically outperforms or matches CCGP in 11 scenarios per metric. Gains are clearest on small- and medium-scale sets; selected large-scale scenarios show additional HV gains. Behavioral and ablation evidence supports preference-dependent rule selection and risk-stratified adjustment.**

***Index Terms*—Cloud–Edge computing, dynamic workflow scheduling, genetic programming, deep reinforcement learning, multi-objective optimization.**

Manuscript received *; revised *; accepted *. This work was supported in part by the Major Key Project of PCL, China under Grant PCL2025A13, and in part by the China Postdoctoral Science Foundation under Grant Number 2026M791634, in part by Shenzhen Science and Technology Program under Grant No. JCYJ20250604145359059, and No. ZDSY20120613125016389. (Corresponding author: Chonglin Gu.)

Zaixing Sun and Bin Wang are with the Department of New Networks, Pengcheng Laboratory, Shenzhen, 518066, China. E-mail: zaixing.sun@micc.hitsz.edu.cn, wangb02@pcl.ac.cn.

Kaixuan Ma, Chonglin Gu and Hejiao Huang are with the Shenzhen Key Laboratory of Internet Information Collaboration, School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen, 518055, China. E-mail: m2018617714@gmail.com, guchonglin@hit.edu.cn, huanghejiao@hit.edu.cn.

Meng Xu is with Beijing Key Laboratory of Lightweight Intelligent System, Beijing Institute of Technology, Beijing, 100081, China. E-mail: mengxu@bit.edu.cn.

I. Introduction

Cloud–Edge computing provides a heterogeneous execution environment for workflow applications by combining latency-proximate Edge Nodes with elastic Cloud Servers. These applications include scientific workflows, data analytics pipelines, Internet of Things (IoT) services, and emerging artificial intelligence (AI) agent orchestration services [1], [2], [3], [4]. Many workflow applications are represented as directed acyclic graphs (DAGs), where tasks are constrained by precedence and data dependencies [1], [2], [3]. As workflows arrive dynamically, a scheduler must order ready tasks and allocate heterogeneous resources without knowing future arrivals. Energy consumption has also become an important system-level constraint. Data centers consumed around 415 TWh worldwide in 2024, accounting for about 1.5% of global electricity demand, and their share of metered electricity consumption reached 23% in Ireland in 2025 [5], [6]. These trends make dynamic workflow scheduling in Cloud–Edge computing a key problem for improving quality of service while controlling resource use and energy consumption.

The main difficulty is that each resource allocation must be evaluated in the current workflow and resource state, because its effects may propagate beyond the task being assigned. *First*, workflow dependencies make task completion times affect downstream execution. The finish time of a task determines when its successors can be released and may further affect workflow completion and service quality. *Second*, Cloud–Edge heterogeneity creates different trade-offs between latency and capacity. Edge Nodes can shorten data paths and avoid Cloud provisioning, but their computing capacity is limited. Cloud Servers provide elastic capacity, but may introduce additional communication and provisioning overheads. *Third*, the CPU-sharing model considered in this paper couples the progress of tasks running on the same resource. Container virtualization provides a practical execution substrate for workflow tasks [7], and modern platforms support runtime resource adjustment through mechanisms such as Kubernetes in-place resource resizing and Docker CPU share or quota updates [8], [9], [10]. In our model, multiple tasks may execute on one resource and share its CPU capacity. Assigning a new task changes the running-task set, so the CPU allocated to CPU-sensitive tasks may be recomputed. This can change remaining execution times, successor release times, resource active duration, and energy consumption. Therefore, an effective scheduler should consider dependency-driven task urgency, heterogeneous resource effects, and runtime CPU allocation when making online decisions. In a multi-objective setting, resource selection should also adapt to the requested tardiness–energy preference.

Many dynamic workflow scheduling studies use manual rules or genetic programming (GP) hyper-heuristics to support online decisions. Manual rules often combine task priorities, sub-deadlines, resource reuse, cost or energy estimates, and candidate filtering to guide task sequencing and resource allocation [11], [12], [13], [14], [15]. Their scheduling logic is explicit, but their performance depends on expert knowledge in selecting and combining task, workflow, and resource attributes. When the execution environment, resource model, workload pattern, or objective preference changes, these rules often require redesign or careful adjustment. GP hyper-heuristics reduce this manual design effort by automatically evolving priority rules from selected attributes. Instead of searching for one schedule for one instance, GP searches the heuristic space and produces rules that can be applied to new dynamic instances. GP has been used to evolve single-tree, multi-objective, multi-tree, dual-tree, and cooperative rules for task selection, resource selection, and related workflow scheduling decisions [16], [17], [18], [19], [20], [21]. However, most GP hyper-heuristic methods deploy one evolved heuristic or a fixed set of cooperating rules throughout an online scheduling run. They do not decide online which evolved resource-selection rule is more suitable for the current task urgency, resource availability, CPU allocation, and objective preference.

Deep reinforcement learning (DRL) offers a way to learn online selection policies from scheduling experience, but two issues must be addressed in this setting. *First*, directly using feasible resources as actions is difficult for dynamic workflow scheduling in Cloud–Edge computing because the action set changes with task requirements, resource availability, and on-demand Cloud provisioning [22], [23], [24]. Rule-assisted DRL avoids this issue by using scheduling rules as fixed actions, where the selected rule ranks feasible candidates at the current decision point. Existing rule-assisted studies use manual rules or GP-evolved rules in dynamic scheduling [25], [26], [27], and GP-evolved actions have also been applied to dynamic Fog workflow scheduling [28]. However, these studies mainly address single-objective or fixed-objective settings, and rule sets based on manual rules still depend on expert design. *Second*, different users or scenarios may require different tardiness–energy trade-offs. Multi-objective reinforcement learning (MORL) and Pareto set learning can condition decisions on preferences [29], [30], [31], [32], [33], [34]. However, they do not provide a rule-mode action design that combines a compact pool of GP-evolved resource-selection rules with preference-aware online selection. These observations motivate combining GP-based generation of complementary resource-selection rules with DRL-based preference-aware online selection.

To address this gap, this paper proposes Preference-aware Risk-Stratified Genetic Programming-assisted Deep Reinforcement Learning (PRS-GPDRL). Cooperative coevolution genetic programming (CCGP) first evolves coupled task-selection and resource-selection rules, so each resource-selection rule is evaluated together with a cooperating task-selection rule. A quality- and behavior-aware extraction procedure then constructs a compact pool of complementary resource-selection rules from the final CCGP archive. Each retained resource-selection rule is paired with a base mode and a risk mode. The base mode follows the selected GP rule, while the risk mode adds a bounded preference-aware local correction. Pairing the retained rules with these two modes gives a fixed rule-mode action space for the DRL agent. A multi-objective double deep Q-network (DDQN) learns to select rule-mode actions according to the current system state and requested preference. Fixed upward-rank ordering sequences ready tasks, allowing the learned policy to focus on resource selection. The main contributions are as follows:

- We propose a CCGP-to-DRL scheduling framework that extracts high-quality and behaviorally complementary resource-selection rules from the CCGP archive. Pairing the resulting compact pool with base and risk modes creates a fixed rule-mode action space whose actions rank the variable set of feasible resources at each allocation decision.
- We introduce risk-stratified action augmentation, in which a GP rule supplies a global resource ranking and the risk mode applies a bounded local correction based on the current state and requested preference. FiLM conditioning, vector-valued action costs, and augmented weighted Tchebycheff scalarization enable one DDQN to respond to different tardiness–energy preferences.
- Across 12 dynamic scheduling scenarios, with 30 independent runs conducted per scenario, PRS-GPDRL obtains the best average ranks for hypervolume (HV, 1.406) and inverted generational distance (IGD, 1.478). It is statistically better than or comparable to CCGP in 11 of 12 scenarios and to GATES and LWFF in at least 11 of 12 scenarios for each metric. The clearest advantages occur on small- and medium-scale workflow sets, with additional HV gains in selected large-scale scenarios.

II. Related Work

A. Dynamic Workflow Scheduling in Cloud–Edge Systems

Dynamic workflow scheduling in Cloud–Edge systems has been studied from both rule-based and learning-based perspectives. Manual rules commonly use task ranks, sub-deadlines, resource reuse, incremental costs, or non-dominated candidate construction for task sequencing and resource allocation [11], [12], [13], [14], [15]. These methods support deadline, energy, cost, or utilization objectives with explicit scheduling logic. Their performance, however, depends on expert knowledge and may require redesign when the execution environment, resource model, workload pattern, or objective preference changes.

Learning-based methods learn scheduling decisions from data or simulation experience. DRL and multi-agent DRL have been applied to time–energy scheduling, multi-objective resource allocation, and collaborative workflow offloading across the Cloud–Edge continuum [22], [23], [35]. GATES uses heterogeneous-graph attention to represent varying ready-task and virtual-machine sets, and learns a direct allocation policy through an evolution strategy [24]. GP hyper-heuristics follow a different route by evolving executable priority rules. CCGP coevolves task-selection, resource-selection, and container-scaling trees and returns non-dominated scheduling heuristics for dynamic Cloud–Fog workflows [21]. These studies provide useful methods for dynamic workflow scheduling. However, they either rely on manual rules, learn direct resource-selection policies, or deploy a fixed GP heuristic after training. They do not focus on preference-aware online selection among GP-evolved resource-selection rules.

B. Rule-Assisted Reinforcement Learning for Dynamic Scheduling

Rule-assisted DRL uses scheduling rules as actions to handle variable candidate sets in dynamic scheduling. Instead of selecting directly from a changing set of jobs or resources, the agent selects a rule from a fixed action set, and the selected rule ranks the feasible candidates. Liu et al. used this indirect action representation with manual dispatching rules in dynamic flexible job shops [25]. This representation accommodates changing candidate sets, but a manual rule library still depends on expert design and may limit the quality and diversity of available actions [26].

Subsequent studies replace manual rules with evolved rules. Niching and behavior clustering preserve different scheduling behaviors, while GP provides executable routing or sequencing actions for online DRL selection [26], [27]. The same principle has also been applied to dynamic Fog workflow scheduling with GP-evolved actions [28]. These studies broaden the source of rule actions through evolution, but they mainly address single-objective scheduling or fixed objective settings. They do not combine a GP-evolved resource-selection rule pool with preference-aware multi-objective action selection for dynamic workflow scheduling in Cloud–Edge computing.

C. Preference-Aware Multi-Objective Learning

Multi-objective reinforcement learning (MORL) addresses sequential decisions with vector-valued outcomes by representing more than one objective trade-off [29], [30]. One group of methods learns explicit policy sets or a continuous Pareto manifold, providing coverage across non-dominated behaviors [36], [37]. Another group incorporates the requested preference into a shared value function or policy. Dynamic weights, desired returns, objective-specific action distributions, and preference-controllable meta-policies are different ways to adapt one learned model without retraining it for every trade-off [31], [32], [38], [39], [40].

Pareto set learning develops a related preference-to-solution view. Augmented Tchebycheff scalarization can train a model that maps preferences to approximate Pareto solutions [33], and PSLGP carries this principle into preference-aware GP heuristics for multi-objective dynamic scheduling [34]. These studies show how a learned model can respond to different objective preferences. However, they do not provide the rule-mode action design needed to let a DRL agent select among GP-evolved resource-selection rules for dynamic workflow scheduling in Cloud–Edge computing.

III. PROBLEM MODELING

We consider a standard distributed and heterogeneous Cloud–Edge computing environment. Let $R = C \cup E$ be the set of heterogeneous computing resources, where C and E are the sets of Cloud Servers and Edge Nodes, respectively. Each resource $r \in R$ is a VM with CPU capacity $\Gamma^c(r)$. Edge Nodes form a fixed resource set with limited computing capacity, whereas Cloud Servers can be provisioned on demand. Tasks are packaged as containers through existing image files, and the containers are deployed to resources. A resource can run multiple containers simultaneously, but their total allocated CPU cannot exceed the CPU capacity of the resource at any time. A container is the minimum execution unit and executes only one task at a time. The scheduling model treats CPU capacity as the adjustable resource.

Let B_{edge} and L_{edge} be the bandwidth and latency for communications within the edge side, respectively. Let B_{cloud} and L_{cloud} be the bandwidth and latency for communications involving Cloud Servers, respectively. Each resource has static power and dynamic power. Static power $p^s(r)$ includes base power $p^b(r)$ and CPU-based idle power $p^{ic}(r)$, where $p^s(r) = p^b(r) + p^{ic}(r)$. Dynamic power is defined as the difference between full power and idle power, including CPU-based power $p^{dc}(r) = p^{fc}(r) - p^{ic}(r)$. The energy consumption of both Cloud Servers and Edge Nodes is considered in this work.

Let $\mathcal{W}$ be the set of workflow applications. Each workflow $\omega \in \mathcal{W}$ contains a set of tasks $A(\omega)$. The arrival time, weight, and deadline of workflow ω are $\rho^a(\omega)$, $\rho^w(\omega)$, and $\rho^d(\omega)$, respectively. Each task $\alpha \in A(\omega)$ is represented by a tuple $\{\phi(\alpha), \tau^l(\alpha), \mu^c(\alpha), A^{pre}(\alpha), A^{suc}(\alpha)\}$, where $\phi(\alpha) \in \{0, 1\}$ denotes the task category, $\tau^l(\alpha)$ is its CPU workload, $\mu^c(\alpha)$ is its minimum CPU requirement, and $A^{pre}(\alpha), A^{suc}(\alpha) \subseteq A(\omega)$ are its predecessor and successor task sets, respectively. $d(\alpha_1, \alpha_2)$ is the data transferred from task α_1 to its successor task $\alpha_2 \in A^{suc}(\alpha_1)$. The execution time of a task depends on its category and the CPU configuration of its container. Tasks fall into the following two categories.

- $\phi(\alpha) = 0$. The container's CPU needs to meet the minimum CPU demand of the task, while improving the container's CPU configuration cannot reduce the execution time of the task. Such tasks are usually CPU-insensitive.
- $\phi(\alpha) = 1$. The container's CPU needs to meet the minimum CPU demand of the task, while improving the container's CPU configuration can reduce the execution time of the task, and vice versa. Such tasks are usually CPU-sensitive.

We formulate the dynamic workflow scheduling problem with workflow deadlines as an optimization problem aiming to minimize the system-oriented total energy consumption and workflow-oriented total weighted tardiness, subject to the following scheduling and resource constraints:

- No task can be executed until all its predecessor tasks have been completed.
- Each task must be executed by a container with sufficient CPU capacity to satisfy its minimum CPU requirement.

- Each resource can deploy and run multiple containers simultaneously, but the sum of their allocated CPU capacities cannot exceed the resource's CPU capacity at any moment.

We assume scheduling as a discrete-event control problem. Let $\mathcal{T}$ be the set of decision points, and $t \in \mathcal{T}$ be a specific decision point. Decision points typically correspond to moments when the system's state changes, such as when a workflow is submitted, a task becomes ready, a task starts execution, or a task is completed. Let $x^s(\alpha)$ and $x^f(\alpha)$ be the execution start time and execution finish time of task α. Let $y(\alpha) \in R$ be the assigned resource of task α in the schedule. Let $\nu^c(\alpha, t)$ be the size of CPU allocated to the container used to execute task α at time t.

The problem can then be formulated as

$$\min \mathbb{E} = \sum_{r \in R} \int_{\mathcal{T}} P(r,t)dt, \tag{1}$$

$$\min \mathbb{T} = \sum_{\omega \in \mathcal{W}} \rho^w(\omega) \times \max\left\{0, \max_{\alpha \in A(\omega)} \left\{x^f(\alpha)\right\} - \rho^d(\omega)\right\}, \tag{2}$$

$$s.t.: x^s(\alpha) \geq \rho^a(\omega),\ \forall\ \omega \in \mathcal{W}, \alpha \in A(\omega), \tag{3}$$

$$x^f(\alpha_1) + x^c(\alpha_1, \alpha_2) + x^l(\alpha_2) \leq x^s(\alpha_2),\ \forall \alpha_2 \in A^{suc}(\alpha_1), \tag{4}$$

$$\begin{cases} x^f(\alpha) - x^s(\alpha) = \dfrac{\tau^l(\alpha)}{\mu^c(\alpha)}, & \phi(\alpha) = 0, \\ \displaystyle\int_{x^s(\alpha)}^{x^f(\alpha)} \nu^c(\alpha, t)dt = \tau^l(\alpha), & \phi(\alpha) = 1. \end{cases} \tag{5}$$

$$\nu^c(\alpha, t) \geq \mu^c(\alpha),\ \forall t \in \mathcal{T}, \tag{6}$$

$$\sum_{\alpha \in A_r(t)} \nu^c(\alpha, t) \leq \Gamma^c(r),\ \forall r \in R, \forall t \in \mathcal{T}, \tag{7}$$

$$x^l(\alpha) = \begin{cases} L_{\text{edge}}, & y(\alpha) \in E, \\ L_{\text{cloud}}, & y(\alpha) \in C. \end{cases} \tag{8}$$

$$x^c(\alpha_1, \alpha_2) = \begin{cases} 0, & y(\alpha_1) = y(\alpha_2), \\ \dfrac{d(\alpha_1, \alpha_2)}{B_{\text{edge}}}, & y(\alpha_1) \neq y(\alpha_2), \\ & y(\alpha_1), y(\alpha_2) \in E, \\ \dfrac{d(\alpha_1, \alpha_2)}{B_{\text{cloud}}}, & \text{otherwise}. \end{cases} \tag{9}$$

The CPU allocation of containers is updated whenever a task starts or completes on a resource. In this work, resource assignment is determined by the scheduling policy, and CPU allocation is then updated according to the following rule. Let $A_r(t)$ be the set of running tasks on resource r at time t, i.e.,

$$A_r(t) = \{\alpha \mid y(\alpha) = r,\ x^s(\alpha) \leq t < x^f(\alpha)\}. \tag{10}$$

Let $S_r(t)$ be the set of CPU-sensitive running tasks on resource r at time t:

$$S_r(t) = \{\alpha \in A_r(t) \mid \phi(\alpha) = 1\}. \tag{11}$$

The remaining CPU capacity after satisfying the minimum CPU requirements of all running tasks is

$$\Gamma^c_{\text{rem}}(r,t) = \Gamma^c(r) - \sum\nolimits_{\beta \in A_r(t)} \mu^c(\beta). \tag{12}$$

For each running task $\alpha \in A_r(t)$, the CPU allocation is calculated as

$$\nu^c(\alpha, t) = \begin{cases} \mu^c(\alpha), & \phi(\alpha) = 0, \\ \mu^c(\alpha) + \dfrac{\Gamma^c_{\text{rem}}(r,t)}{|S_r(t)|}, & \phi(\alpha) = 1. \end{cases} \tag{13}$$

Thus, CPU-insensitive tasks always use their minimum CPU requirements, while CPU-sensitive tasks equally share the remaining CPU capacity.

The CPU utilization of resource r at time t is

$$U^c(r,t) = \frac{\sum_{\alpha \in A_r(t)} \nu^c(\alpha, t)}{\Gamma^c(r)}. \tag{14}$$

The instantaneous power consumption of resource r is modeled as

$$P(r,t) = p^s(r) + p^{dc}(r) \left(U^c(r,t)\right)^3. \tag{15}$$

The above equations hold for $\forall\ \omega \in \mathcal{W}$, $\forall\ \alpha, \alpha_1, \alpha_2 \in A(\omega)$, $\forall\ r \in R$, and $\forall t \in \mathcal{T}$ when the corresponding quantities are defined.

Eqs. (1) and (2) are the two considered objectives, respectively. Eq. (1) gives the total energy consumption by integrating the instantaneous power $P(r,t)$ over all Cloud Servers and Edge Nodes. Eq. (2) is the workflow-oriented total weighted tardiness. Constraint (3) ensures that the workflow cannot be executed until it is submitted. Constraint (4) defines workflow precedence. A task can start only after all immediate predecessors have finished and their output data have arrived. Eq. (5) governs the two task categories and ensures that task α completes workload $\tau^l(\alpha)$ during $[x^s(\alpha), x^f(\alpha)]$ under the corresponding CPU allocation. Constraint (6) ensures that a container's CPU allocation satisfies the task's minimum requirement. Constraint (7) ensures that allocated container CPU does not exceed resource capacity at time t. Eq. (8) defines the network latency $x^l(\alpha)$, which depends on whether the task is assigned to an Edge Node or a Cloud Server. Eq. (9) gives the communication time between task α_1 and its successor α_2. Eq. (10) defines the set of running tasks on a resource. Eq. (11) defines the set of CPU-sensitive running tasks on a resource. Eq. (12) calculates the remaining CPU capacity after the minimum requirements of all running tasks have been satisfied. Eq. (13) defines the CPU allocation of containers after task-start or task-completion events. Eq. (14) is the CPU-based resource utilization. Eq. (15) models the instantaneous power consumption $P(r,t)$ of a resource r at time t. The power consists of the static power $p^s(r)$ and the CPU-based dynamic power, which scales cubically with CPU utilization. For on-demand Cloud Servers, Eq. (15) is evaluated only over recorded utilization intervals of provisioned resources. Unprovisioned Cloud Servers do not contribute to the energy integral in Eq. (1).

IV. The Proposed PRS-GPDRL Algorithm

A. Overall Framework of PRS-GPDRL

Fig. 1 shows the overall framework, which contains offline training and online scheduling. For each training scenario, CCGP produces the final CCGP archive, from which action extraction retains a compact resource-selection rule pool $\mathcal{G}$.

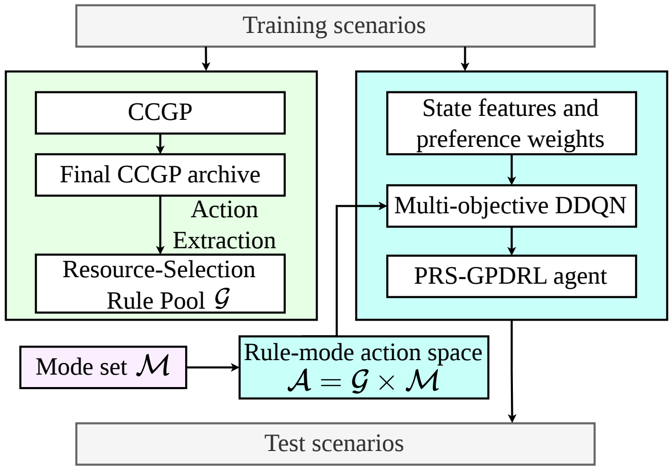


Fig. 1. Overall framework of PRS-GPDRL.

The base and risk modes form the mode set $\mathcal{M}$; pairing the two sets gives the rule-mode action space $\mathcal{A}$. Using this action space, the state features, and the preference weights, the multi-objective DDQN is trained to form the PRS-GPDRL agent.

During online scheduling, when multiple tasks are simultaneously ready, fixed upward-rank ordering sequences them [41]. For the task currently being scheduled, the PRS-GPDRL agent first selects a resource-selection rule and mode; the selected action then scores the feasible resources and assigns the task to the resource with the lowest priority.

B. CCGP Training and Action Extraction

CCGP training. The CCGP stage follows the cooperative coevolution mechanism used for tree-based scheduling rules. CCGP maintains two subpopulations: one for tree-based task-selection rules and the other for tree-based resource-selection rules. During cooperative evaluation, an individual in one subpopulation is paired with a representative from the other. The resulting complete tree-based scheduling heuristic is evaluated using energy consumption and weighted tardiness. During CCGP evolution, evaluated complete scheduling heuristics are recorded in an archive. Until the stopping criterion is met, CCGP updates the subpopulations by elitism, reproduction, crossover, and mutation. At the final generation, duplicate solutions are removed from the archive set (AS), and its first non-dominated front is retained as the candidate set for subsequent resource-selection rule extraction.

Rule representation. The tree-based rules are represented by terminal and function nodes. Terminal nodes provide scheduling attributes observed from the current workflow, task, candidate resource, and system state, while function nodes combine these values to compute priority scores. The task- and resource-selection rules use different terminal subsets and share the same function set. Table I summarizes the terminals used by the CCGP rules and the corresponding attributes used by the DRL state encoder. The GP and DDQN columns indicate how each attribute is used by the two components. Further details of CCGP training and the computation of GP terminal values are given in [21].

TABLE I
TERMINAL, FUNCTION, AND STATE-ENCODER ATTRIBUTES USED BY PRS-GPDRL.

Notation	Description	GP	DDQN
CAT	Category of a task.	T	T
SD	Sub-deadline of a task.	T	T
ET	Execution time of a task on a reference resource.	T	T
CPUA	Minimum CPU requirement of a task.	T	T
WT	Weight of a workflow.	T	T
UR	Task upward-rank value.	T	T
NSA	Number of direct successors of a task.	T	T
NPA	Number of direct predecessors of a task.	T	–
NKQ	Number of tasks in the ready queue.	T	–
TETQ	Total ET of tasks in the ready queue.	T	–
NRA	Number of remaining tasks in a workflow.	T	T
TETRA	Total ET of remaining workflow tasks.	T	T
VCT	Average communication time for a task.	T	T
NEW	Whether the candidate resource is newly created.	–	R
WAIT	Waiting time before the task starts on the candidate resource.	R	R
ETAR	Minimum execution time of a task on a resource.	R	R
CRCPU	Current remaining CPU of a resource.	R	R
ST	Slack time of a task on a resource.	R	R
STPR	Static power of a resource.	R	R
DPR	Dynamic power of a candidate resource.	R	R
NTR	Number of tasks on a resource.	R	R
NKW	Number of scheduled tasks from the same workflow in the Cloud or Edge tier.	R	R
CPUR	CPU capacity of a resource.	R	R
CTR	Communication time of a task on a resource.	R	R
w_T	Preference weight for weighted tardiness.	–	P
w_E	Preference weight for energy consumption.	–	P
Function set	$+$, $-$, $\times$, protected /, Max, and Min.	T/R	–

In the GP column, T and R denote task- and resource-selection rule terminals. In the DDQN column, T, R, and P denote task, candidate-resource, and preference attributes, respectively.

Action extraction. After CCGP training, PRS-GPDRL extracts a compact pool of resource-selection rules represented as GP trees from the candidate set retained from AS. Action extraction reduces the rule-pool size while preserving diversity, thereby lowering the difficulty of subsequent DRL training. Let g_i denote candidate resource-selection rule i. The extraction procedure constructs $\mathcal{G}$ as follows.

Validation evaluation. Each candidate is evaluated on a validation set of random seeds to obtain a coarse bi-objective performance estimate for anchor selection. Its validation performance is represented by the mean energy consumption and mean weighted tardiness over three independent runs. Let $S_i^{\mathrm{val}} = \widehat{\mathbb{E}}_i^{\mathrm{val}} + \widehat{\mathbb{T}}_i^{\mathrm{val}}$ denote the normalized validation score of individual i, where $\widehat{\mathbb{E}}_i^{\mathrm{val}}$ and $\widehat{\mathbb{T}}_i^{\mathrm{val}}$ are its min–max normalized validation energy and tardiness, respectively.

Decision-point sampling. A reference CCGP individual is selected from the validation-evaluated candidates by minimizing S_i^{val}. The reference individual is then executed to collect representative resource-allocation decision points. At each sampled decision point, all feasible resources are recorded with their current terminal attributes.

Behavioral representation. For each candidate resource-selection rule, its behavior is measured by replaying it on the sampled decision points. At decision point d, the rule scores all feasible resources and selects the resource with the best priority. The behavior vector of g_i is

$$\mathbf{b}_i = (\pi_i(d_1), \pi_i(d_2), \ldots, \pi_i(d_M)), \tag{16}$$

where $\pi_i(d_m)$ is the resource index selected by g_i at sampled decision point d_m, and $M = 20$ is the number of sampled decision points. In implementation, $\pi_i(d_m)$ is encoded as a resource-selection indicator vector, so tied best resources are represented as the same selected set. Rules producing identical behavior vectors are grouped together, and the representative minimizing S_i^{val} is retained.

Anchor selection. Let K denote the maximum requested rule-pool size. If the number of behaviorally unique rules does not exceed K, all representatives are retained. For the default $K = 5$, the validation front otherwise supplies the minimum-energy endpoint, a low-energy neighbor, a balanced-knee rule, a low-tardiness neighbor, and the minimum-tardiness endpoint. The endpoints minimize $\widehat{\mathbb{E}}_i^{\text{val}}$ and $\widehat{\mathbb{T}}_i^{\text{val}}$, respectively. After excluding the endpoints, each neighbor is drawn from the corresponding 5% normalized objective band, i.e., its corresponding normalized value does not exceed 0.05; leading candidates under that objective ranking are also considered when the band is sparse. Let i_E and i_T denote the two endpoint indices. The balanced anchor is

$$\begin{aligned} S_i^{\text{bal}} &= \max\left\{\widehat{\mathbb{E}}_i^{\text{val}}, \widehat{\mathbb{T}}_i^{\text{val}}\right\} + 0.05\left(\widehat{\mathbb{E}}_i^{\text{val}} + \widehat{\mathbb{T}}_i^{\text{val}}\right), \\ i_{\text{bal}} &= \arg\min_{i \notin \{i_E, i_T\}} S_i^{\text{bal}}. \end{aligned} \tag{17}$$

For the $K = 3$ ablation, the minimum-energy endpoint, the balanced-knee rule, and the minimum-tardiness endpoint are retained. If an anchor candidate duplicates an already selected behavior, the next behaviorally distinct candidate from the same region or objective ranking is used.

For a given CCGP run b, the retained resource-selection rules form the compact GP rule pool $\mathcal{G} = \{g_1, \ldots, g_{K_b}\}$, where $K_b \le K$ is the actual number retained. Combining $\mathcal{G}$ with the two modes gives the rule-mode action space $\mathcal{A}$.

C. Risk-Stratified Action Augmentation

The extracted GP rule pool provides complementary global resource-scoring behaviors, and the DDQN selects among their complete GP-induced priority rankings at rule level. PRS-GPDRL augments each rule with *base* and *risk* modes. The selected resource-selection rule provides the base priorities, while the risk mode adds a bounded, preference-aware local correction. The resulting action space combines rule-level behavior selection with local preference adaptation over feasible resources.

Let $\mathbf{w} = (w_T, w_E)$ be the current preference vector, where w_T and w_E are the weights for weighted tardiness and energy consumption, respectively, and $w_T + w_E = 1$. At a resource-selection decision point, let $\mathcal{R}_t$ be the set of feasible resources and let $n = |\mathcal{R}_t|$. For a selected resource-selection rule $g_i \in \mathcal{G}$, let $\hat{g}_i(r)$ denote its min–max normalized priority score for resource $r \in \mathcal{R}_t$. For all min–max normalizations in this work, if all resources have the same value, the corresponding normalized score is set to zero for all resources.

PRS-GPDRL then estimates two objective-specific local costs for each resource in the same feasible-resource set. For the tardiness-oriented local cost, let α_t be the current ready task at decision time t, let $\mathsf{SD}(\alpha_t)$ be its sub-deadline, and let ω be the workflow containing α_t. For resource r, let $t_{\alpha_t,r}^{\text{ava}}$ be the actual available time for assigning α_t to r. An allocation snapshot is computed by inserting α_t into r at $t_{\alpha_t,r}^{\text{ava}}$ and applying the CPU allocation rule in Eq. (13). Let $\hat{x}_{\alpha_t,r}^{f}$ be the estimated finish time of α_t in this snapshot. The tardiness-oriented local cost combines the estimated time from the current decision point to task completion with the workflow-weighted tardiness relative to the sub-deadline:

$$c_T(r) = \hat{x}_{\alpha_t,r}^{f} - t + \rho^w(\omega)\max\{0, \hat{x}_{\alpha_t,r}^{f} - \mathsf{SD}(\alpha_t)\}, \tag{18}$$

For the energy-oriented local cost, PRS-GPDRL compares the resource state before and after assigning the current task. Let U_r^0 and U_r^1 be its CPU utilization before and after assignment, and let H_r^0 and H_r^1 be the corresponding busy horizons measured from $t_{\alpha_t,r}^{\text{ava}}$ to its tail completion time. Using the power terms defined in Section III, the dynamic energy increment is

$$\Delta E_r^{\text{dyn}} = p^{dc}(r)\left((U_r^1)^3 H_r^1 - (U_r^0)^3 H_r^0\right). \tag{19}$$

ΔE_r^{dyn} is a lightweight estimate of the dynamic-energy increment caused by assigning the current task to resource r. The static energy increment depends on whether the resource is newly created, already active, or idle:

$$\Delta E_r^{\text{sta}} = \begin{cases} p^s(r) H_r^1, & \text{new}, \\ p^s(r)\left(t_r^{\text{tail},1} - t_r^{\text{tail},0}\right), & \text{active}, \\ p^s(r)\left(t_{\alpha_t,r}^{\text{ava}} - t_r^{\text{last}} + H_r^1\right), & \text{idle}, \end{cases} \tag{20}$$

where $t_r^{\text{tail},0}$ and $t_r^{\text{tail},1}$ are the tail completion times before and after assignment, and t_r^{last} is the last utilization event time of r. For an idle resource, the static-energy increment includes the idle interval since its last utilization event and the new busy horizon caused by assigning the current task. The energy-oriented local cost is

$$c_E(r) = \Delta E_r^{\text{dyn}} + \Delta E_r^{\text{sta}}. \tag{21}$$

Let $\hat{c}_T(r)$ and $\hat{c}_E(r)$ be the min–max normalized objective-specific local costs used to make tardiness and energy comparable within the current feasible-resource set. A preference-weighted auxiliary score is then defined as

$$c_{\mathbf{w}}(r) = w_T \hat{c}_T(r) + w_E \hat{c}_E(r). \tag{22}$$

The auxiliary score ranks the feasible resources for risk stratification. Let $\operatorname{rank}_{\mathbf{w}}(r) \in \{1, \ldots, n\}$ be the ascending rank of $c_{\mathbf{w}}(r)$. PRS-GPDRL maps the rank to four risk strata:

$$\chi_{\mathbf{w}}(r) = \begin{cases} 0.00, & \operatorname{rank}_{\mathbf{w}}(r) \le \lceil 0.10n \rceil, \\ 0.20, & \lceil 0.10n \rceil < \operatorname{rank}_{\mathbf{w}}(r) \le \lceil 0.25n \rceil, \\ 0.50, & \lceil 0.25n \rceil < \operatorname{rank}_{\mathbf{w}}(r) \le \lceil 0.50n \rceil, \\ 1.00, & \operatorname{rank}_{\mathbf{w}}(r) > \lceil 0.50n \rceil. \end{cases} \tag{23}$$

Risk denotes the relative undesirability indicated by the auxiliary ranking. A lower risk level indicates that the resource is ranked closer to the preferred local choice under the requested preference. The cutoffs define top, near-top, medium-risk, and high-risk resource groups, respectively.

For the base mode, the risk term is disabled. For the risk mode, the mixing coefficient is

$$\beta_{\mathrm{risk}}(\mathbf{w}) = \beta_{\max}|w_T - w_E|^2, \tag{24}$$

Here, $|w_T - w_E|$ measures the departure from the balanced preference. When $w_T = w_E = 0.5$, the risk correction is disabled and the risk mode reduces to the base mode. Its influence increases smoothly toward either objective endpoint, reaching $\beta_{\max}$ at an extreme preference.

The final priority score of the resource-selection rule g_i under mode m is

$$p_m(r; g_i) = \begin{cases} \hat{g}_i(r), & m = \mathrm{base}, \\ \hat{g}_i(r) + \beta_{\mathrm{risk}}(\mathbf{w})\chi_{\mathbf{w}}(r), & m = \mathrm{risk}. \end{cases} \tag{25}$$

For a selected rule g^* and mode m^*, the selected resource is $r_t^* = \arg\min_{r \in \mathcal{R}_t} p_{m^*}(r; g^*)$.

D. MDP Formulation for Preference-Aware Rule Selection

The resource-selection problem is formulated as an MDP over task-ready decision points. It is represented by $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathbf{c}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the rule-mode action space, $\mathcal{P}$ is the state-transition kernel, $\mathbf{c}$ is the vector cost, and γ is the discount factor.

At decision point t, state $s_t \in \mathcal{S}$ contains the task, candidate-resource, and preference attributes marked in the DDQN column of Table I. Task and preference features are fixed-length, whereas the variable-size candidate-resource list is padded and accompanied by a validity mask.

Let $\mathcal{M} = \{\mathrm{base}, \mathrm{risk}\}$ be the mode set. The action space is

$$\mathcal{A} = \mathcal{G} \times \mathcal{M}, \qquad a_t = (g_i, m), \qquad g_i \in \mathcal{G},\ m \in \mathcal{M}. \tag{26}$$

An actual pool size of K_b therefore yields $2K_b$ rule-mode actions in run b. Executing a_t applies the selected rule and mode to the feasible resources as defined in Section IV-C and assigns the current task to r_t^*. The scheduling process then advances to the next task-ready decision point or episode termination, producing the next state and transition cost.

Each MDP transition supplies the two-dimensional immediate cost used to train the DDQN. Let ΔT_t and ΔE_t denote the tardiness-related cost and energy increment, respectively, produced by the simulator transition from one decision point to the next. The stored cost is

$$\mathbf{c}_t = (\Delta T_t, \kappa \Delta E_t), \tag{27}$$

where κ aligns the numerical scales of the two components during training; evaluation uses the original objective values.

E. Preference-Aware Multi-Objective DDQN

PRS-GPDRL uses a multi-objective DDQN [42] to select rule-mode actions. At each decision point, the observation is first encoded into a state representation and then fed to a vector-valued DDQN. The resulting two-objective action-cost estimates are scalarized by the current preference for action selection and DDQN updates.

Fig. 2 illustrates the state encoder and rule-mode action selection. The raw observation s_t contains task features, candidate-resource features, and preference weights. The state encoder maps s_t to embedding $\mathbf{z}_t$, and the DDQN estimates vector action costs over $\mathcal{A}$. Augmented weighted Tchebycheff scalarization selects $a_t^* = (g^*, m^*)$, where $g^* \in \mathcal{G}$ and $m^* \in \mathcal{M}$ are the selected rule and mode. Algorithm 1 applies them to score the feasible resources and assign ready task α_t to r_t^*.

Let $\mathbf{x}_t^{\mathrm{task}}$, $\mathbf{X}_t^{\mathrm{res}}$, and $\mathbf{w}$ denote the task features, candidate-resource features, and preference vector, respectively. Continuous task and resource attributes are transformed using signed logarithmic scaling, while categorical indicators remain unchanged. The task encoder comprises two 64-unit layers, each followed by an ELU activation. Feasible resources are separated into Edge Node and Cloud Server sets. Each tier-specific resource encoder contains two 64-unit ELU layers that project resource features into key representations, followed by a 64-dimensional linear value projection. Single-head masked scaled dot-product attention uses the task embedding as the query and the projected resource representations as keys to aggregate the corresponding values into a 64-dimensional tier-level representation. The task embedding and the two tier-level representations are concatenated and passed through two 128-unit fusion layers, with an ELU activation after the first, to produce $\mathbf{h}_{\mathrm{fuse}} \in \mathbb{R}^{128}$.

A feature-wise linear modulation (FiLM) layer [43] modulates the fused representation according to the preference. A two-layer preference MLP maps the two preference weights through a 32-unit ELU hidden layer to a 256-dimensional output, which is split into 128-dimensional raw-scale and shift vectors. The scale is parameterized as $\boldsymbol{\gamma}_{\mathrm{film}}(\mathbf{w}) = 1 + 0.3\tanh(\widetilde{\boldsymbol{\gamma}}_{\mathrm{film}}(\mathbf{w}))$, which bounds its elements between 0.7 and 1.3 and prevents multiplicative sign reversal or excessive amplification; the shift remains unconstrained. The FiLM transformation is

$$\mathbf{z}_t = \boldsymbol{\gamma}_{\mathrm{film}}(\mathbf{w}) \odot \mathbf{h}_{\mathrm{fuse}} + \boldsymbol{\delta}_{\mathrm{film}}(\mathbf{w}), \tag{28}$$

where $\boldsymbol{\gamma}_{\mathrm{film}}(\mathbf{w})$ and $\boldsymbol{\delta}_{\mathrm{film}}(\mathbf{w})$ are preference-dependent scale and shift vectors. The resulting $\mathbf{z}_t \in \mathbb{R}^{128}$ is the DDQN state embedding.

The 128-dimensional state embedding is processed by a shared two-layer MLP with hidden sizes 256 and 128 and ReLU activations. A linear output layer then produces a two-dimensional cost estimate for every rule-mode action:

$$\mathbf{Q}_\theta(s_t, a) = \left(Q_\theta^T(s_t, a), Q_\theta^E(s_t, a)\right), \quad a \in \mathcal{A}. \tag{29}$$

Here, Q_θ^T and Q_θ^E estimate cumulative tardiness and energy costs, respectively, preserving both dimensions for preference-dependent scalarization.

Since lower values are better for both objectives, action selection minimizes an augmented weighted Tchebycheff cost. For state s, the estimated ideal component for objective j is $q_\theta^{j,\min}(s) = \min_{a' \in \mathcal{A}} Q_\theta^j(s, a')$, where $j \in \{T, E\}$. The scalarized cost is

$$\begin{aligned} \psi_{\mathbf{w}}(\mathbf{Q}_\theta(s,a)) = {} & \max_{j \in \{T,E\}} w_j \Big(Q_\theta^j(s,a) - q_\theta^{j,\min}(s) \Big) \\ & + \zeta \sum_{j \in \{T,E\}} w_j \Big(Q_\theta^j(s,a) - q_\theta^{j,\min}(s) \Big), \end{aligned} \tag{30}$$

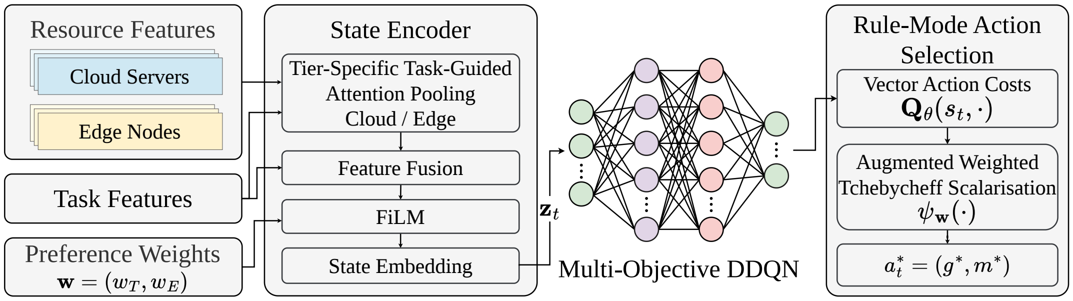


Fig. 2. Multi-objective DDQN for rule-mode action selection in PRS-GPDRL.

Algorithm 1: Online preference-aware resource selection.

Input: State s_t containing preference $\mathbf{w}$; ready task α_t; feasible resource set $\mathcal{R}_t$

Output: Selected resource r_t^*

Encode s_t and evaluate $\mathbf{Q}_\theta(s_t, a)$ for all $a \in \mathcal{A}$;
Select $a_t^* = (g^*, m^*)$ by Eqs. (30) and (31);
Evaluate and normalize $g^*(r)$ over all $r \in \mathcal{R}_t$;
if $m^* = \mathrm{base}$ **then** Return $r_t^* = \arg\min_{r \in \mathcal{R}_t} \hat{g}^*(r)$;
Estimate the normalized costs $\hat{c}_T(r)$ and $\hat{c}_E(r)$ over $\mathcal{R}_t$;
Construct the auxiliary score $c_\mathbf{w}(r)$ and the four-level $\chi_\mathbf{w}(r)$ by Eqs. (22) and (23);
Compute $\beta_{\mathrm{risk}}(\mathbf{w})$ by Eq. (24);
Return $r_t^* = \arg\min_{r \in \mathcal{R}_t} p_{\mathrm{risk}}(r; g^*)$ using Eq. (25);

where ζ is the augmentation coefficient. The estimated ideal point gives objective-wise deviations from the best predicted action costs, and the augmentation term reduces ties. During training, Eq. (30) is the preference-conditioned scalar objective used for the DDQN update. During action selection, this scalarization determines the greedy action:

$$a_t^* = \arg\min_{a \in \mathcal{A}} \psi_\mathbf{w}\left(\mathbf{Q}_\theta(s_t, a)\right). \tag{31}$$

The vector action-cost function is trained using Double DQN [42] with experience replay and a periodically updated target network. At the start of each training episode, a preference vector is sampled from the discrete grid

$$w_T \in \{0, 0.05, \ldots, 0.95, 1\}, \qquad w_E = 1 - w_T, \tag{32}$$

included in the state, and fixed throughout the episode. Checkpoint validation evaluates each validation episode under all 21 grid preferences. For preference $\mathbf{w}$, the cumulative validation cost is $C^{\mathrm{val}}(\mathbf{w}) = w_T \sum_t \Delta T_t + w_E \kappa \sum_t \Delta E_t$. The checkpoint with the lowest mean $C^{\mathrm{val}}(\mathbf{w})$ across the validation episodes and grid preferences is retained. The remaining training parameters are reported in Table IV.

F. Training and Inference Procedure

After CCGP training and action extraction in Section IV-B, the action space is fixed for offline DRL training. A scenario is denoted by $\langle \mathrm{wfNum}, \lambda \rangle$, where wfNum is the number of submitted workflows and λ is their arrival rate. Training and testing are performed separately for each scenario using the fixed action space. For the sampled preference, the DDQN learns from vector-cost transitions at ready-task decision points using the scalar objective in Eq. (30). The trained DDQN and fixed rule-mode action space constitute the PRS-GPDRL agent.

During inference, the PRS-GPDRL agent uses the DDQN to select a rule-mode action and assigns α_t to the feasible resource with the lowest corresponding priority. Scheduling then proceeds to the next task-ready decision point.

V. Performance Evaluation and Results

A. Baselines and Performance Measures

PRS-GPDRL is compared with CCGP, GATES, and LWFF.

- **CCGP** [21] coevolves task-selection, resource-selection, and container-scaling GP rules to construct non-dominated scheduling heuristics. For the problem model in Section III, our implementation retains its cooperative task- and resource-rule evolution but omits the container-scaling subpopulation because container CPU is assigned by Eq. (13). Each combined heuristic is evaluated on energy and weighted tardiness, and the final non-dominated heuristics form the approximation set.
- **GATES** [24] uses graph attention to encode ready tasks, workflow dependencies, and candidate resources, and evolves a neural resource-selection policy through an evolution strategy. Our implementation retains this graph representation and direct resource-selection policy. To support the two objectives, policy networks are evaluated on energy and weighted tardiness, NSGA-II performs individual selection, and offspring networks are generated by Gaussian perturbations of the selected network parameters. The final non-dominated policies form the approximation set.
- **LWFF** [15] is a constructive heuristic that selects resources from non-dominated candidates according to resource waste and completion speed. In our implementation, ready tasks are ordered by upward rank, and the energy and tardiness estimates of feasible resources are min–max normalized and combined under each requested preference. Expected finish time resolves ties. Applying LWFF over the evaluation preference grid and retaining

the non-dominated schedules produces its approximation set.

PRS-GPDRL uses the resource-selection rule pool extracted from the corresponding CCGP run. In each run, one non-dominated approximation set is obtained for each method: solutions over the preference grid for PRS-GPDRL and LWFF, and the final heuristics or policies for CCGP and GATES. LWFF produces the same approximation set in every run; its non-zero HV and IGD standard deviations arise because this set is jointly normalized with those of the other methods in each run. The four sets are pooled, and each objective $f_j \in \{\mathbb{T}, \mathbb{E}\}$ is min–max normalized as

$$\widehat{f}_j(x) = \frac{f_j(x) - f_j^{\min}}{f_j^{\max} - f_j^{\min}}, \tag{33}$$

where $f_j^{\min}$ and $f_j^{\max}$ are the pooled extrema of the four sets in that run; a constant objective is assigned zero.

HV evaluates the convergence and coverage of each approximation set and is computed against the reference point $(1, 1)$. Because the true Pareto front is unknown, the IGD reference set Q comprises the jointly non-dominated normalized solutions from the four sets in that run. For approximation set P,

$$\mathrm{IGD}(P, Q) = \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|p - q\|_2. \tag{34}$$

Larger HV and smaller IGD are preferable.

For each scenario and metric, the four algorithms are compared using the Iman–Davenport-corrected Friedman test and two-sided paired Wilcoxon signed-rank tests at a significance level of 0.05. Within each result cell, the arrows denote statistically better (↑), worse (↓), or similar (≈) performance against the preceding methods. For each baseline column, W/D/L gives the number of scenarios in which that baseline is statistically better than, similar to, or worse than PRS-GPDRL; the PRS-GPDRL column is therefore marked N/A. AverageRank is the mean rank over the 30 runs, for which smaller is better.

B. Experimental Setup

Experiments use the event-driven workflow simulator introduced in [21]. Allocation updates occur at the task events specified in Section III. All methods use the same workflow instances, feasible resource set, communication model, CPU allocation rule, and energy accounting.

Table II lists five representative resource types for each tier. The configurations are derived from AWS EC2[1], and the power parameters follow the Teads model[2]. The Edge tier contains 19 fixed Edge Nodes, whereas Cloud Servers of the listed types can be provisioned on demand. Each task incurs a 50-ms container initialization delay [7], and provisioning a new Cloud Server takes 55.9 s. Following [44], [45], LAN and WAN bandwidths are set to 2.0 and 0.5 GB/s, and their communication latencies are 0.5 and 30 ms, respectively.

Workflow templates are sampled from Alibaba Cluster-trace-v2018 [3]. The public trace supplies task dependencies and execution information, from which the DAG workflows used by the simulator are reconstructed. Task input and output data sizes follow a discrete uniform distribution between 500 and 5000 MB. Each task is independently assigned as CPU-sensitive or CPU-insensitive with equal probability. Workflow weights are independently sampled from $\{1, 2, 4\}$ with probabilities 0.2, 0.6, and 0.2, respectively, following [46].

Workflow arrivals follow a Poisson process with rate λ, so the inter-arrival time is exponentially distributed with mean $1/\lambda$ [47]. Following [21], reference execution and Edge transfer times are propagated along each DAG. Let L_ω be the largest earliest finish time obtained by this propagation. The workflow deadline is $\rho^d(\omega) = \rho^a(\omega) + \delta_\omega L_\omega$, where the deadline factor δ_ω is sampled uniformly from $\{0.8, 1.0, 1.5, 1.8\}$.

Training and test protocol.

Each scenario is denoted by $\langle \mathrm{wfNum}, \lambda \rangle$, where wfNum is the total number of submitted workflows and λ is their arrival rate. The main evaluation covers the 12 combinations of $\mathrm{wfNum} \in \{10, 15, 20, 30, 50, 100\}$ and $\lambda \in \{0.2, 0.8\}$. For each scenario, 30 independent runs are conducted. Within each run, every solution is evaluated on the same 30 unseen test instances, and its objective values are averaged over these instances before HV and IGD are calculated. The tables report the mean and standard deviation of HV and IGD over the 30 runs.

The parameter settings of CCGP follow [21] and are presented in Table III. The DDQN and risk-stratified action augmentation configuration is detailed in Table IV. GATES uses a population of 100 policy networks and runs for 51 generations.

We set $K = 5$. Behavioral filtering may leave fewer than five unique rules in a particular run; the actual pool size in run b is denoted by $K_b \leq K$. Each retained rule is paired with the two modes, so that run b has $2K_b$ nominal rule-mode actions. Training preference sampling, checkpoint validation, and evaluation use the grid in Eq. (32).

C. Solution-Set Quality and Pareto Trade-offs

1) HV Results: Tables V and VI present the mean and standard deviation of the HV and IGD values, respectively, over the 30 independent runs of the compared algorithms.

PRS-GPDRL obtains the best HV average rank (1.406), while CCGP records 1/1/10 relative to it. HV gains are clearest at $\mathrm{wfNum} = 10$. PRS-GPDRL is statistically better than or comparable to CCGP throughout $\mathrm{wfNum} \leq 50$. At $\mathrm{wfNum} = 100$, the outcome depends on the arrival rate: an additional gain occurs at $\langle 100, 0.2 \rangle$, but not at $\langle 100, 0.8 \rangle$, where the maximum workflow scale and higher arrival rate coincide.

2) IGD Results: PRS-GPDRL also obtains the best IGD average rank (1.478), with CCGP recording 1/4/7 relative to it. IGD improvements are again clearest at $\mathrm{wfNum} = 10$. PRS-GPDRL is statistically better than or comparable to CCGP throughout $\mathrm{wfNum} \leq 50$, whereas the $\mathrm{wfNum} = 100$

[1] https://aws.amazon.com/ec2/

[2] https://engineering.teads.com/sustainability/carbon-footprint-estimator-for-aws-instances/

[3] https://github.com/alibaba/clusterdata

TABLE II
CONFIGURATIONS OF EDGE NODES AND CLOUD SERVERS.

Edge Nodes					Cloud Servers				
Type (number)	ECU	CPU idle power (W)	CPU full power (W)	Base power (W)	Type	ECU	CPU idle power (W)	CPU full power (W)	Base power (W)
m1.small (3)	1	0.72	5.76	1.2	m4.xlarge	13	1.94	15.62	3.2
m3.medium (3)	3	0.87	6.97	1.4	m4.2xlarge	26	3.89	31.24	6.4
m3.large (4)	6.5	1.73	13.94	2.9	m4.4xlarge	53.5	7.77	62.47	12.9
m3.xlarge (4)	13	3.47	27.87	5.8	m4.10xlarge	124.5	24.12	193.88	40.0
m3.2xlarge (5)	26	6.94	55.74	11.5	m4.16xlarge	188	31.09	249.90	51.6

TABLE III
THE PARAMETER SETTINGS OF CCGP.

Parameter	Value
Population size	2 subpopulations of 50
Number of generations	51
Method for initializing population	Ramped-half-and-half
Initial minimum/maximum depth	2/7
Elitism	5
Maximal depth	8
Crossover/mutation/reproduction rate	0.80/0.15/0.05
Terminal/non-terminal selection rate	10%/90%
Environmental selection	NSGA-II

TABLE IV
THE PARAMETER CONFIGURATION OF DDQN AND RISK-STRATIFIED ACTION AUGMENTATION.

Parameter	Value
Training-step budget	1,000,000
Exploration rate ϵ	1.0 decays to 0.01
Discount factor γ	1.0
Learning rate	5×10^{-5}
Optimizer	Adam
Mini-batch size	256
Replay memory size	200,000
Target-update frequency	5,000
DDQN hidden-layer sizes	256/128
Maximum rule-pool size K	5
Maximum risk mixing $\beta_{\max}$	0.75
Energy-cost scaling κ	4.7×10^5
Tchebycheff augmentation ζ	0.05

TABLE V
THE MEAN AND STANDARD DEVIATION OF THE HV VALUES OVER THE 30 INDEPENDENT RUNS OF THE COMPARED ALGORITHMS.

$\langle \mathrm{wfNum}, \lambda \rangle$	GATES	LWFF	CCGP	PRS-GPDRL
$\langle 10, 0.2 \rangle$	0.533±0.189	0.597±0.131(≈)	0.789±0.092(↑)(↑)	0.867±0.068(↑)(↑)(↑)
$\langle 10, 0.8 \rangle$	0.536±0.211	0.616±0.124(↑)	0.788±0.113(↑)(↑)	0.882±0.059(↑)(↑)(↑)
$\langle 15, 0.2 \rangle$	0.566±0.124	0.602±0.107(≈)	0.800±0.114(↑)(↑)	0.862±0.087(↑)(↑)(↑)
$\langle 15, 0.8 \rangle$	0.539±0.128	0.651±0.109(↑)	0.749±0.150(↑)(↑)	0.843±0.081(↑)(↑)(↑)
$\langle 20, 0.2 \rangle$	0.607±0.168	0.632±0.122(≈)	0.789±0.134(↑)(↑)	0.869±0.088(↑)(↑)(↑)
$\langle 20, 0.8 \rangle$	0.530±0.170	0.646±0.107(↑)	0.776±0.110(↑)(↑)	0.825±0.060(↑)(↑)(↑)
$\langle 30, 0.2 \rangle$	0.635±0.139	0.675±0.091(↑)	0.823±0.101(↑)(↑)	0.867±0.060(↑)(↑)(↑)
$\langle 30, 0.8 \rangle$	0.568±0.132	0.697±0.091(↑)	0.788±0.091(↑)(↑)	0.803±0.090(↑)(↑)(≈)
$\langle 50, 0.2 \rangle$	0.611±0.152	0.702±0.098(↑)	0.808±0.103(↑)(↑)	0.851±0.114(↑)(↑)(↑)
$\langle 50, 0.8 \rangle$	0.644±0.164	0.749±0.103(↑)	0.783±0.135(↑)(↑)	0.815±0.110(↑)(↑)(↑)
$\langle 100, 0.2 \rangle$	0.579±0.192	0.697±0.090(↑)	0.817±0.096(↑)(↑)	0.847±0.077(↑)(↑)(↑)
$\langle 100, 0.8 \rangle$	0.798±0.081	0.866±0.059(↑)	0.852±0.067(↑)(↓)	0.810±0.086(≈)(↓)(↓)
W/D/L	0/1/11	1/0/11	1/1/10	N/A
AverageRank	3.603	3.017	1.975	1.406

TABLE VI
THE MEAN AND STANDARD DEVIATION OF THE IGD VALUES OVER THE 30 INDEPENDENT RUNS OF THE COMPARED ALGORITHMS.

$\langle \mathrm{wfNum}, \lambda \rangle$	GATES	LWFF	CCGP	PRS-GPDRL
$\langle 10, 0.2 \rangle$	0.321±0.135	0.235±0.081(↑)	0.100±0.032(↑)(↑)	0.047±0.024(↑)(↑)(↑)
$\langle 10, 0.8 \rangle$	0.310±0.145	0.199±0.063(↑)	0.107±0.038(↑)(↑)	0.042±0.020(↑)(↑)(↑)
$\langle 15, 0.2 \rangle$	0.257±0.099	0.166±0.054(↑)	0.068±0.031(↑)(↑)	0.040±0.028(↑)(↑)(↑)
$\langle 15, 0.8 \rangle$	0.303±0.094	0.154±0.054(↑)	0.084±0.057(↑)(↑)	0.047±0.034(↑)(↑)(↑)
$\langle 20, 0.2 \rangle$	0.260±0.128	0.173±0.077(↑)	0.078±0.041(↑)(↑)	0.050±0.048(↑)(↑)(↑)
$\langle 20, 0.8 \rangle$	0.276±0.133	0.151±0.075(↑)	0.074±0.035(↑)(↑)	0.055±0.038(↑)(↑)(≈)
$\langle 30, 0.2 \rangle$	0.247±0.110	0.152±0.050(↑)	0.083±0.037(↑)(↑)	0.053±0.046(↑)(↑)(↑)
$\langle 30, 0.8 \rangle$	0.291±0.108	0.127±0.039(↑)	0.080±0.030(↑)(↑)	0.074±0.043(↑)(↑)(≈)
$\langle 50, 0.2 \rangle$	0.232±0.095	0.112±0.040(↑)	0.082±0.042(↑)(↑)	0.059±0.043(↑)(↑)(↑)
$\langle 50, 0.8 \rangle$	0.240±0.105	0.099±0.032(↑)	0.074±0.039(↑)(↑)	0.075±0.041(↑)(↑)(≈)
$\langle 100, 0.2 \rangle$	0.269±0.139	0.146±0.063(↑)	0.075±0.034(↑)(↑)	0.079±0.040(↑)(↑)(≈)
$\langle 100, 0.8 \rangle$	0.190±0.078	0.079±0.055(↑)	0.066±0.024(↑)(≈)	0.135±0.056(↑)(↓)(↓)
W/D/L	0/0/12	1/0/11	1/4/7	N/A
AverageRank	3.792	2.847	1.883	1.478

outcomes depend on the arrival rate. The adapted GATES and LWFF baselines are statistically worse than or comparable to PRS-GPDRL in at least 11 scenarios per metric.

The CCGP comparison controls GP-rule provenance: PRS-GPDRL extracts its rule pool from the corresponding CCGP run, so both methods share the same source of GP resource-selection knowledge. Their difference therefore cannot be explained by a different or better GP-rule source. Together with the behavior and ablation results, this pattern supports online selection among complementary rules and state- and preference-dependent local adjustment.

3) 50% Empirical Attainment Curves: Fig. 3 presents the 50% empirical attainment curves of the compared algorithms in the normalized weighted-tardiness–energy space for six representative scenarios.

PRS-GPDRL lies below CCGP over most of their shared tardiness range in five panels, with the clearest separation in the two $\mathrm{wfNum} = 10$ scenarios. Because the separation spans the trade-off interior, the HV and IGD gains reflect broad set-quality improvements rather than a few endpoint solutions.

D. Preference-Conditioned Policy Behavior

1) Objective Response to Preference: Fig. 4 shows the scenario-wise median relative reductions in weighted tardiness and energy consumption as their corresponding preference weights vary across the 12 scenarios. Let $x(w_j)$ denote the solution obtained at objective weight w_j. For objective f_j, the relative reduction is $[f_j(x(0)) - f_j(x(w_j))]/f_j(x(0))$ from the endpoint at which its own weight is zero. Weighted tardiness is plotted against w_T, and energy against $w_E = 1 - w_T$.

At the outcome level, increasing an objective's weight changes the schedule in the intended direction. Median reductions at weight 0.5 are 89.7% for weighted tardiness and 20.5% for energy; over the full range, they reach 96.9% and 51.1%.

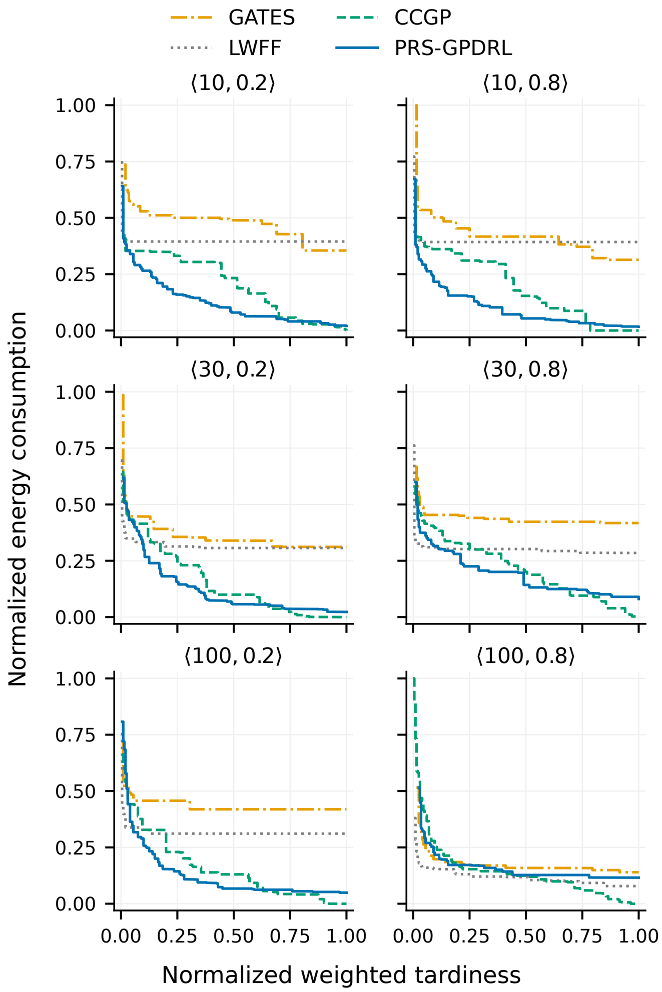


Fig. 3. The 50% empirical attainment curves of the compared algorithms in six representative scenarios.

The steeper tardiness response is consistent with urgency-oriented decisions avoiding deadline penalties immediately, whereas energy savings accumulate through assignments that alter utilization and resource active time.

The median correlations between each raw objective and its own weight are $\rho_T = -0.968$ and $\rho_E = -0.955$. All tardiness curves and 99.4% of energy curves have the expected direction; 99.4% of runs also shift correctly between the endpoints. Thus, one network produces preference-dependent scheduling outcomes rather than relabeling the same outcome with different weights.

2) Preference-Dependent Resource-Selection Rule Usage: Fig. 5 shows how the use of the five retained resource-selection rules varies with tardiness preference across the 12 scenarios. The rules are ordered from the minimum-energy endpoint to the minimum-tardiness endpoint.

At the rule level, the minimum-energy rule and its neighbor account for 61.7% of decisions at $w_T = 0$, while the minimum-tardiness pair accounts for 50.3% at $w_T = 1$. The median weight–role correlation is $\rho_g = 0.738$, and 90.4% of included runs shift as expected. This explains Fig. 4 internally: increasing w_T moves selection from energy-oriented roles through the knee toward tardiness-oriented roles.

3) Risk-Mode Usage: Fig. 6 shows the risk-mode selection frequency as a function of w_T for the two arrival rates, with one panel for each workflow count.

At the mode level, the risk mode is selected in 42.5% of decisions. Away from balance, non-zero correction is activated in 41.2%, ranging from 26.1% to 56.6% across preferences and scenarios. The risk mode is therefore not rarely selected, while the remaining base-mode decisions show that local correction is not always required. The policy adaptively chooses between using the unmodified GP ranking and applying a local correction as the state and preference vary.

Figs. 5 and 6 capture two distinct levels of adaptation. Rule-role migration sets the global objective direction of the resource ranking, moving from energy-oriented roles through the knee toward tardiness-oriented roles as w_T increases. Risk-mode selection instead gates whether the current decision requires a bounded local modification of that ranking. Its frequency need not vary monotonically with w_T, because the need for correction depends jointly on the requested preference, task urgency, CPU sharing, resource active time, Cloud provisioning, and the current feasible-resource state. Risk-mode frequency therefore reflects state- and preference-dependent gating behavior rather than serving as a direct proxy for the preference weight.

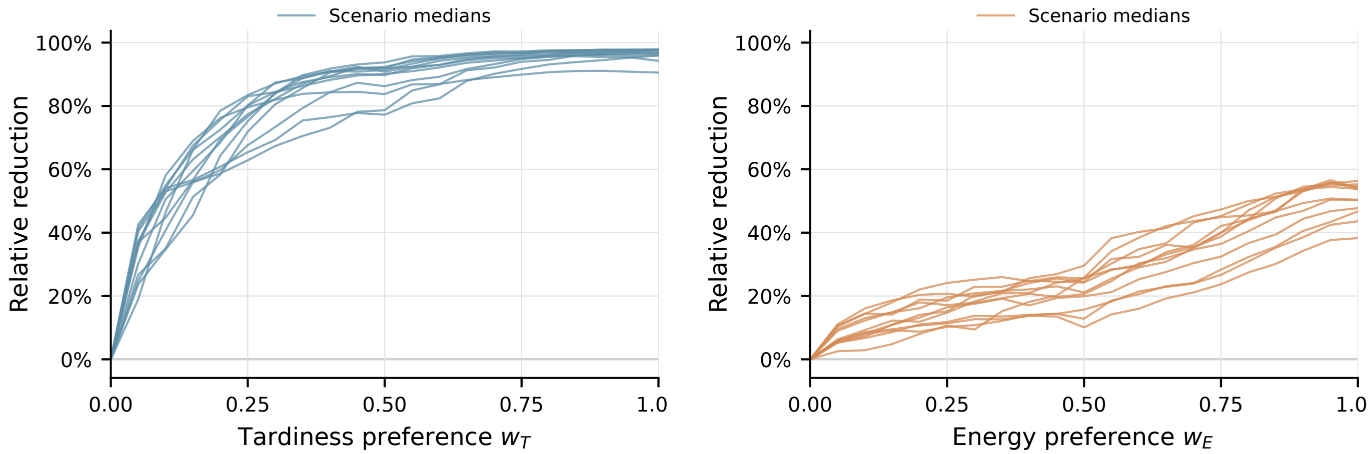
Within the evaluated preference grid, the three figures form a layered account of policy behavior. Fig. 4 shows preference response at the outcome level; Fig. 5 shows the corresponding migration of global rule roles; and Fig. 6 shows whether local correction is selected at the mode level. Taken together, these observations support the interpretation that preference conditioning changes both the resulting objectives and the internal decisions that produce them: which global ranking is used and whether it is locally corrected for the current state.

E. Ablation Analysis

Table VII reports the HV and IGD results of PRS-GPDRL and its three ablation variants in six representative scenarios. The *w/o Risk* and *w/o FiLM* variants remove the two components separately, while $K = 3$ sets the maximum requested rule-pool size to three. The evaluation and statistical analysis follow the main protocol.

1) Risk-Stratified Action Augmentation and FiLM Conditioning: Across the six representative scenarios, *w/o Risk* records 0/0/6 against PRS-GPDRL for both metrics, the strongest ablation effect. A GP rule provides a global ranking; Risk uses the specified preference, task urgency, CPU sharing, resource active time, and Cloud provisioning for bounded local correction. The consistent result across these six scenarios supports state- and preference-dependent local correction as an important part of the full performance improvement.

By comparison, the *w/o FiLM* variant records 0/3/3 for both metrics and retains the vector-valued action costs and augmented weighted Tchebycheff scalarization, so preference information remains available to action evaluation. The three statistically comparable outcomes and three losses therefore suggest that FiLM is not the only preference-conditioning



(a) Weighted tardiness response to w_T.

(b) Energy response to w_E.

Fig. 4. Relative reductions in weighted tardiness and energy consumption under different preference weights. Each curve shows the median over 30 independent runs for one scenario.

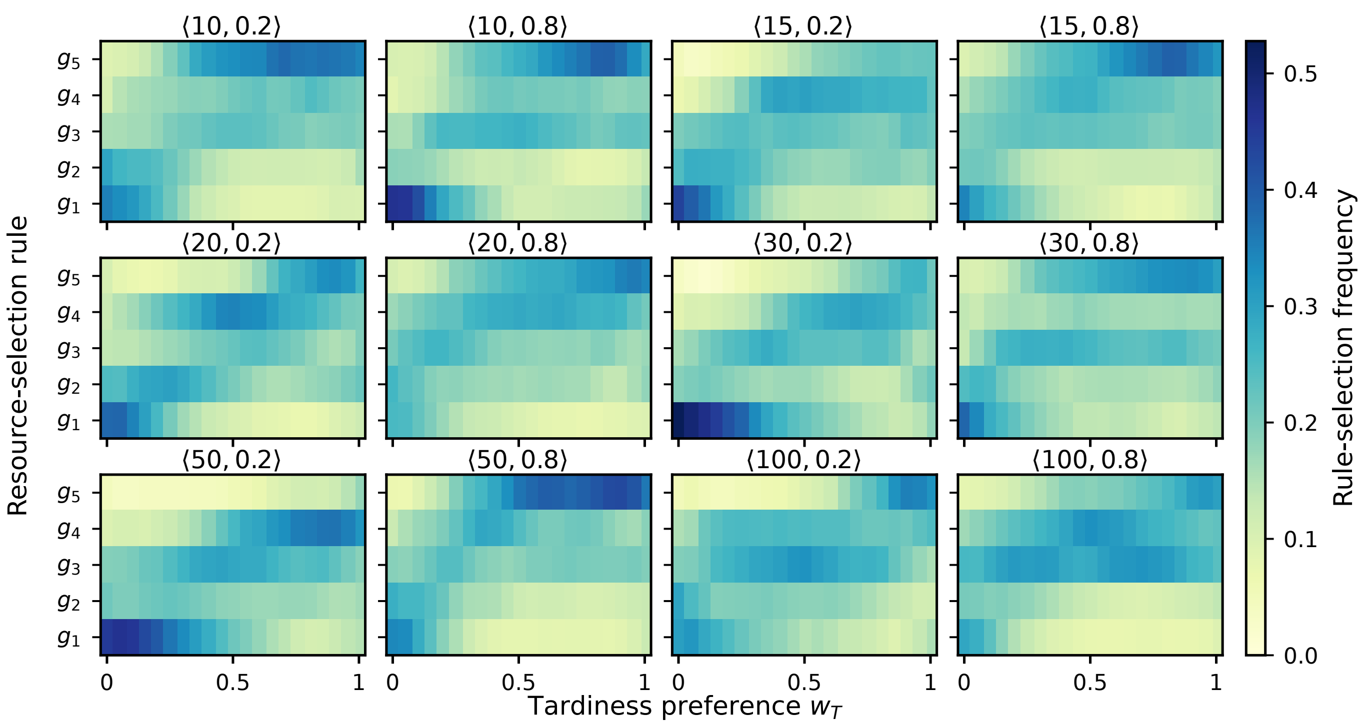


Fig. 5. Mean selection frequencies over the included independent runs for each of the 12 scenarios under different tardiness preferences (runs retaining five rules). The roles are the minimum-energy endpoint g_1, its neighbor g_2, the knee g_3, the minimum-tardiness neighbor g_4, and the minimum-tardiness endpoint g_5.

channel. Instead, its additional preference-dependent modulation of the shared state representation is consistent with complementary improvements in trade-off coverage and proximity.

2) Rule-Pool Size: The $K = 3$ result is 0/5/1 for both metrics and is statistically comparable in five of six scenarios. Thus, the main capability does not arise simply from increasing the number of available actions; a small set of behaviorally complementary rules retains most of it. Within the tested settings, performance is reasonably robust to pool size when behaviorally diverse rule roles are retained. The $K = 5$ design targets both endpoints, their neighboring regions, and the knee, and obtains the best average ranks. This pattern is consistent with the motivation for quality- and behavior-aware filtering: coverage of complementary global-ranking roles is more important than rule count alone.

The ablations thus organize the components into distinct, complementary roles: Risk provides state- and preference-dependent local correction, FiLM adds preference-dependent modulation of the shared representation, and the rule pool provides complementary global-ranking coverage. These roles

TABLE VII
THE MEAN AND STANDARD DEVIATION OF THE HV AND IGD VALUES FOR PRS-GPDRL AND ITS ABLATION VARIANTS OVER 30 INDEPENDENT RUNS.

$\langle$wfNum, $\lambda\rangle$	HV				IGD			
	w/o Risk	w/o FiLM	$K = 3$	PRS-GPDRL	w/o Risk	w/o FiLM	$K = 3$	PRS-GPDRL
$\langle 10, 0.2\rangle$	.832±.098	.851±.076(≈)	.860±.071(↑)(↑)	.867±.068(↑)(↑)(≈)	.078±.046	.058±.030(↑)	.052±.033(↑)(≈)	.047±.024(↑)(↑)(≈)
$\langle 10, 0.8\rangle$	.827±.093	.855±.068(↑)	.880±.064(↑)(↑)	.882±.059(↑)(↑)(≈)	.076±.047	.058±.032(↑)	.039±.017(↑)(↑)	.042±.020(↑)(↑)(≈)
$\langle 30, 0.2\rangle$	.814±.103	.848±.074(↑)	.851±.076(≈)(≈)	.867±.060(↑)(↑)(↑)	.090±.058	.068±.047(↑)	.068±.057(↑)(≈)	.053±.046(↑)(↑)(↑)
$\langle 30, 0.8\rangle$	.752±.105	.814±.087(↑)	.809±.081(↑)(≈)	.803±.090(↑)(≈)(≈)	.114±.059	.065±.032(↑)	.074±.034(↑)(≈)	.074±.043(↑)(≈)(≈)
$\langle 100, 0.2\rangle$	.778±.117	.828±.075(↑)	.854±.055(↑)(↑)	.847±.077(↑)(≈)(≈)	.118±.071	.084±.047(↑)	.082±.042(↑)(≈)	.079±.040(↑)(≈)(≈)
$\langle 100, 0.8\rangle$	.748±.108	.801±.090(↑)	.816±.081(↑)(≈)	.810±.086(↑)(≈)(≈)	.176±.065	.125±.052(↑)	.121±.051(↑)(≈)	.135±.056(↑)(≈)(≈)
W/D/L	0/0/6	0/3/3	0/5/1	N/A	0/0/6	0/3/3	0/5/1	N/A
AverageRank	3.222	2.511	2.164	2.103	3.322	2.389	2.231	2.058

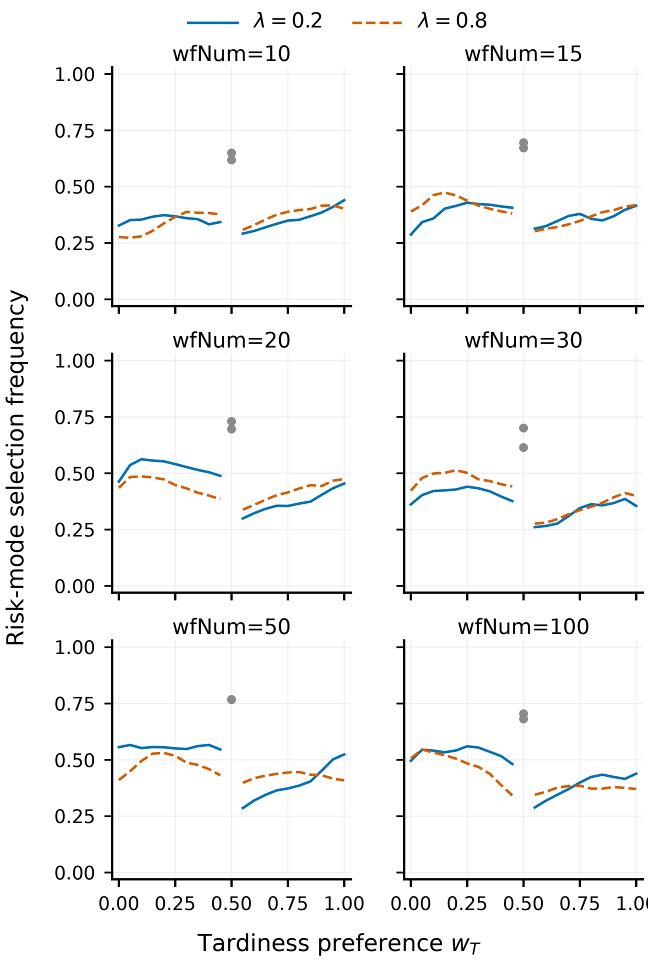


Fig. 6. Mean risk-mode selection frequency over 30 independent runs for each of the 12 scenarios. Gray markers denote the balanced preference $w_T = w_E = 0.5$. At the balanced preference, risk-mode selections are counted although their priority scores coincide with those of the base mode.

are complementary within the proposed decision process.

F. Task-Ordering Design Choice

Table VIII reports the HV and IGD results obtained using the GP task-selection rule and upward-rank ordering across the 12 scenarios. The comparison holds the deduplicated resource-selection rule pool fixed. Following Tables V–VII, W/D/L reports the baseline GP task-selection rule relative to upward-rank ordering.

TABLE VIII
THE MEAN AND STANDARD DEVIATION OF THE HV AND IGD VALUES OBTAINED USING GP TASK-SELECTION RULE AND UPWARD-RANK ORDERING OVER 30 INDEPENDENT RUNS.

$\langle$wfNum, $\lambda\rangle$	HV		IGD	
	GP task rule	Upward rank	GP task rule	Upward rank
$\langle 10, 0.2\rangle$	.789±.092	.790±.092(≈)	.100±.032	.100±.033(≈)
$\langle 10, 0.8\rangle$	.788±.113	.787±.117(≈)	.107±.038	.108±.042(≈)
$\langle 15, 0.2\rangle$	.800±.114	.799±.109(≈)	.068±.031	.070±.028(≈)
$\langle 15, 0.8\rangle$	.749±.150	.750±.151(≈)	.084±.057	.085±.059(≈)
$\langle 20, 0.2\rangle$	.789±.134	.787±.136(≈)	.078±.041	.078±.040(≈)
$\langle 20, 0.8\rangle$	.776±.110	.779±.108(≈)	.074±.035	.072±.033(≈)
$\langle 30, 0.2\rangle$	.823±.101	.823±.100(≈)	.083±.037	.081±.037(≈)
$\langle 30, 0.8\rangle$	.788±.091	.784±.099(≈)	.080±.030	.081±.030(≈)
$\langle 50, 0.2\rangle$	.808±.103	.806±.104(≈)	.082±.042	.084±.042(↓)
$\langle 50, 0.8\rangle$	.783±.135	.779±.136(↓)	.074±.039	.075±.037(≈)
$\langle 100, 0.2\rangle$	.817±.096	.813±.096(≈)	.075±.034	.078±.035(≈)
$\langle 100, 0.8\rangle$	.852±.067	.850±.070(≈)	.066±.024	.067±.026(≈)
W/D/L	1/11/0	N/A	1/11/0	N/A
AverageRank	1.436	1.564	1.458	1.542

With the resource-selection rule pool fixed, the GP task-selection rule records 1/11/0 against upward-rank ordering for both metrics, with similar average ranks. Upward-rank ordering is therefore statistically comparable in 11 of 12 scenarios. This diagnostic result indicates that task-ordering change is not the main explanation for the overall gain over CCGP. This comparability is consistent with the problem structure: VMs can execute tasks concurrently, and task ordering is required only when multiple tasks become ready simultaneously, whereas resource selection is required for every dispatched task. Task ordering is therefore less influential than resource selection in this problem.

Taken together, the experiments narrow the plausible explanation for the overall gain. The V-C comparison controls GP-rule provenance because PRS-GPDRL uses the rule pool extracted from the corresponding CCGP run, while the present comparison indicates that task-ordering change is not the main explanation. The rule-usage analysis shows that the policy changes how this same-source GP knowledge is deployed as preference varies, and the mode-usage analysis shows that local correction is selected conditionally rather than always or never. The $K = 3$ comparison further weakens a simple action-count explanation, while the *w/o Risk* result supports the importance of local correction for the full improvement in the six ablation scenarios. These results jointly support the interpretation that PRS-GPDRL gains value from preference-conditioned online deployment of same-source, behaviorally complementary GP rules, together with bounded local correction when indicated by the current state.

VI. Conclusion

This paper studies preference-aware online scheduling of dynamic workflows in heterogeneous Cloud–Edge systems, with total weighted tardiness and energy as the two objectives. PRS-GPDRL connects evolutionary rule discovery with online reinforcement learning: CCGP evolves coupled scheduling rules, a diversity-aware procedure extracts a compact resource-selection rule pool, and a multi-objective DDQN selects risk-stratified rule-mode actions at task-allocation decision points. This design lets one network adapt the use of GP-evolved resource priorities to the current state and requested objective trade-off.

Across 12 scenarios, with 30 independent runs conducted per scenario, PRS-GPDRL obtains the best average ranks of 1.406 for HV and 1.478 for IGD and is statistically better than or comparable to CCGP in 11 of 12 scenarios for each metric. The clearest advantages occur on small- and medium-scale workflow sets, with additional HV gains in selected large-scale scenarios. The behavior analysis shows preference response at three levels: the objective values, the roles of the selected resource-selection rules, and the choice between using an unmodified GP ranking and applying a local risk correction. Across the six representative ablation scenarios, Risk provides the strongest component-level improvement, FiLM adds complementary preference conditioning, and the $K = 3$ results retain most of the rule-pool capability. The task-ordering experiment further shows that upward-rank ordering is statistically comparable to the GP task-selection rule in 11 of the 12 scenarios. Overall, PRS-GPDRL generates high-quality tardiness–energy trade-offs while adapting its internal scheduling decisions to the requested preference.

References

[1] S. Li, W. Wu, H. Zhang, Y. Liu, W. Lin, and K. Li, "Revisiting workflow scheduling with the power of edge computing: Taxonomy, review, and open challenges," *Comput. Sci. Rev.*, vol. 60, p. 100887, 2026.

[2] R. Bouabdallah and F. Fakhfakh, "Workflow scheduling in cloud–fog computing environments: A systematic literature review," *Concurr. Comput. Pract. Exp.*, vol. 36, no. 28, p. e8304, 2024.

[3] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, "WfCommons: A framework for enabling scientific workflow research and development," *Future Gener. Comput. Syst.*, vol. 128, pp. 16–27, 2022.

[4] Gartner, "Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026, up from less than 5% in 2025," 2025, accessed: Jul. 27, 2026. [Online]. Available: https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025

[5] International Energy Agency, "Energy and AI," 2025, accessed: Jul. 27, 2026. [Online]. Available: https://www.iea.org/reports/energy-and-ai

[6] Central Statistics Office, "Data centres metered electricity consumption 2025," 2026, accessed: Aug. 18, 2026. [Online]. Available: https://www.cso.ie/en/releasesandpublications/ep/p-dcmec/datacentresmeteredelectricityconsumption2025/

[7] R. Ranjan, I. S. Thakur, G. S. Aujla, N. Kumar, and A. Y. Zomaya, "Energy-efficient workflow scheduling using container-based virtualization in software-defined data centers," *IEEE Trans. Ind. Informat.*, vol. 16, no. 12, pp. 7646–7657, 2020.

[8] Kubernetes, "Resize CPU and memory resources assigned to containers," 2026, accessed: Jul. 27, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/

[9] Docker Inc., "docker container update," accessed: Jul. 27, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/update/

[10] ——, "Resource constraints," accessed: Jul. 27, 2026. [Online]. Available: https://docs.docker.com/engine/containers/resource_constraints/

[11] G. Fan, X. Chen, Z. Li, H. Yu, and Y. Zhang, "An energy-efficient dynamic scheduling method of deadline-constrained workflows in a cloud environment," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3089–3103, 2023.

[12] Z. Sun, H. Huang, Z. Li, and C. Gu, "Energy-efficient real-time multi-workflow scheduling in container-based cloud," *J. Comb. Optim.*, vol. 49, no. 2, pp. 1–21, 2025.

[13] K. Li, "Energy-constrained DAG scheduling on edge and cloud servers with overlapped communication and computation," *J. Grid Comput.*, vol. 22, no. 3, p. 60, 2024.

[14] J. Lou, Z. Tang, S. Zhang, W. Jia, W. Zhao, and J. Li, "Cost-effective scheduling for dependent tasks with tight deadline constraints in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5829–5845, 2023.

[15] H. M. Fard, R. Prodan, and F. Wolf, "Dynamic multi-objective scheduling of microservices in the cloud," in *Proc. IEEE/ACM Int. Conf. Utility Cloud Comput.*, 2020, pp. 386–393.

[16] K.-R. Escott, H. Ma, and G. Chen, "A genetic programming hyper-heuristic approach to design high-level heuristics for dynamic workflow scheduling in cloud," in *Proc. IEEE Symp. Ser. Comput. Intell.*, 2020, pp. 3141–3148.

[17] Y. Yu, T. Shi, H. Ma, and G. Chen, "A genetic programming-based hyper-heuristic approach for multi-objective dynamic workflow scheduling in cloud environment," in *Proc. IEEE Congr. Evol. Comput.*, 2022, pp. 1–8.

[18] M. Xu, Y. Mei, S. Zhu, B. Zhang, T. Xiang, F. Zhang, and M. Zhang, "Genetic programming for dynamic workflow scheduling in fog computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 4, pp. 2657–2671, 2023.

[19] Z. Sun, Y. Mei, F. Zhang, H. Huang, C. Gu, and M. Zhang, "Multi-tree genetic programming hyper-heuristic for dynamic flexible workflow scheduling in multi-clouds," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2687–2703, 2024.

[20] Y. Yang, G. Chen, H. Ma, S. Hartmann, and M. Zhang, "Dual-tree genetic programming with adaptive mutation for dynamic workflow scheduling in cloud computing," *IEEE Trans. Evol. Comput.*, vol. 29, no. 5, pp. 1692–1706, 2025.

[21] Z. Sun, F. Zhang, Y. Mei, H. Huang, C. Gu, B. Wang, and M. Zhang, "Cooperative coevolution genetic programming for dynamic joint workflow scheduling and container scaling in cloud-fog computing," *IEEE Trans. Serv. Comput.*, vol. 19, no. 1, pp. 225–239, 2026.

[22] A. Jayanetti, S. Halgamuge, and R. Buyya, "Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge–cloud computing environments," *Future Gener. Comput. Syst.*, vol. 137, pp. 14–30, 2022.

[23] B. Xiao, C. Yu, X. Chen, Z. Chen, and G. Min, "Multi-agent collaboration for workflow task offloading in end-edge-cloud environments using deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 11, pp. 2281–2296, 2025.

[24] Y. Shen, G. Chen, H. Ma, and M. Zhang, "GATES: Cost-aware dynamic workflow scheduling via graph attention networks and evolution strategy," in *Proc. 34th Int. Joint Conf. Artif. Intell.*, 2025, pp. 8635–8643.

[25] R. Liu, R. Piplani, and C. Toro, "Deep reinforcement learning for dynamic scheduling of a flexible job shop," *Int. J. Prod. Res.*, vol. 60, no. 13, pp. 4049–4069, 2022.

[26] M. Xu, Y. Mei, F. Zhang, and M. Zhang, "Niching genetic programming to learn actions for deep reinforcement learning in dynamic flexible scheduling," *IEEE Trans. Evol. Comput.*, vol. 30, no. 1, pp. 61–75, 2026.

[27] F. Zhao, C. Zhao, L. Wang, and H. Sang, "A deep reinforcement learning framework assisted by genetic programming for dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, 2025.

[28] Y. He, Z. Xu, J. Lin, Y. Li, and S. Zhang, "Deep reinforcement learning with evolved actions for dynamic workflow scheduling in distributed fog computing," *Neurocomputing*, vol. 677, p. 133115, 2026.

[29] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, “A survey of multi-objective sequential decision-making,” *J. Artif. Intell. Res.*, vol. 48, pp. 67–113, 2013.
[30] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Reymond, T. Verstraeten, L. M. Zintgraf, R. Dazeley, F. Heintz, E. Howley, A. A. Irissappane, P. Mannion, A. Nowé, G. Ramos, M. Restelli, P. Vamplew, and D. M. Roijers, “A practical guide to multi-objective reinforcement learning and planning,” *Auton. Agents Multi-Agent Syst.*, vol. 36, no. 1, p. 26, 2022.
[31] A. Abels, D. M. Roijers, T. Lenaerts, A. Nowé, and D. Steckelmacher, “Dynamic weights in multi-objective deep reinforcement learning,” in *Proc. 36th Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 11–20.
[32] R. Yang, X. Sun, and K. Narasimhan, “A generalized algorithm for multi-objective reinforcement learning and policy adaptation,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
[33] X. Lin, Z. Yang, X. Zhang, and Q. Zhang, “Pareto set learning for expensive multi-objective optimization,” in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 19 231–19 247.
[34] M. Xu, Y. Mei, F. Zhang, Y. S. Ong, and M. Zhang, “Pareto set learning through genetic programming for multiobjective dynamic scheduling,” *IEEE Trans. Evol. Comput.*, vol. 30, no. 3, pp. 942–956, 2026.
[35] J. Zhang, Z. Ning, M. Waqas, H. Alasmary, S. Tu, and S. Chen, “Hybrid edge-cloud collaborator resource scheduling approach based on deep reinforcement learning and multiobjective optimization,” *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 192–205, 2024.
[36] K. Van Moffaert and A. Nowé, “Multi-objective reinforcement learning using sets of Pareto dominating policies,” *J. Mach. Learn. Res.*, vol. 15, pp. 3663–3692, 2014.
[37] S. Parisi, M. Pirotta, and M. Restelli, “Multi-objective reinforcement learning through continuous Pareto manifold approximation,” *J. Artif. Intell. Res.*, vol. 57, pp. 187–227, 2016.
[38] A. Abdolmaleki, S. Huang, L. Hasenclever, M. Neunert, F. Song, M. Zambelli, M. Martins, N. Heess, R. Hadsell, and M. Riedmiller, “A distributional view on multi-objective policy optimization,” in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 119, 2020, pp. 11–22.
[39] M. Reymond, E. Bargiacchi, and A. Nowé, “Pareto conditioned networks,” in *Proc. 21st Int. Conf. Auton. Agents Multiagent Syst.*, 2022, pp. 1110–1118.
[40] Y. Yang, T. Zhou, M. Pechenizkiy, and M. Fang, “Preference controllable reinforcement learning with advanced multi-objective optimization,” in *Proc. 42nd Int. Conf. Mach. Learn.*, vol. 267, 2025, pp. 71 612–71 647.
[41] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.
[42] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
[43] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3942–3951.
[44] M. Goudarzi, H. Wu, M. Palaniswami, and R. Buyya, “An application placement technique for concurrent IoT applications in edge and fog computing environments,” *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1298–1311, 2021.
[45] S. Pallewatta, V. Kostakos, and R. Buyya, “QoS-aware placement of microservices-based IoT applications in fog computing environments,” *Future Gener. Comput. Syst.*, vol. 131, pp. 121–136, 2022.
[46] T. Hildebrandt and J. Branke, “On using surrogates with genetic programming,” *Evol. Comput.*, vol. 23, no. 3, pp. 343–367, 2015.
[47] S. Sahoo, B. Sahoo, and A. K. Turuk, “A learning automata-based scheduling for deadline sensitive task in the cloud,” *IEEE Trans. Serv. Comput.*, vol. 14, no. 6, pp. 1662–1674, 2021.